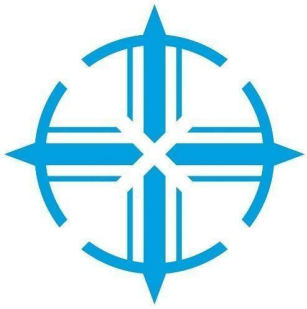
PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR



DESARROLLO EN PLATAFORMAS

T1S7: Depuración, Jest, ESLint, rendimiento, refactorización y documentación técnica

Integrantes:

Ariel Israel Mejía Zambrano, Lenin Marcelo Moreno Guerrero, Ricardo Andrés Arias Barberan, Kevin Javier Montatixe Caiza

Docente:

Damián Aníbal Nicolalde Rodríguez

Quito – Ecuador

17 de Octubre del 2026

1. Introducción

Mejorar una plataforma exige evidencia. Antes de modificar el código hay que reproducir el problema, medir el comportamiento actual y registrar qué cambió; sin una línea base no se puede demostrar que la aplicación quedó más confiable o más rápida. Este informe documenta esa revisión sobre la plataforma de incidentes: dos defectos reproducidos con las herramientas del navegador, cinco pruebas unitarias sobre las reglas de transición, el análisis estático del código, tres mediciones de rendimiento comparadas antes y después, y la revisión de la estructura por capas.

Tabla 1. Actividades de la revisión, evidencia producida y resultado

Actividad	Evidencia	Resultado
Depuración	Punto de interrupción, cuerpo de la petición y respuesta	Dos defectos reproducidos y corregidos (Figuras 2 a 4)
Prueba	Cinco casos y reporte de cobertura	Reglas verificables sin servidor (Figura 5)
Análisis	Reporte del analizador estático	Tres hallazgos atendidos (Figuras 6 y 7)
Rendimiento	Medición antes y después	Transferencia reducida un 87.6 % (Figuras 8 a 10)

Nota. Elaboración propia a partir de Pontificia Universidad Católica del Ecuador (s. f.).

2. Depuración de defectos

El primer defecto permitía cerrar un incidente sin documentar la resolución técnica, lo que rompía la trazabilidad del registro. Se reprodujo enviando el estado de cierre con el campo vacío y con espacios en blanco. Un punto de interrupción en el manejador del formulario mostró que el valor llegaba al servidor y que ninguna regla lo examinaba. La causa estaba en el módulo de transiciones, que validaba el estado de destino pero no la justificación. Se añadió la comprobación de contenido mínimo tras recortar espacios: la entrada inválida ahora recibe un código de solicitud incorrecta y la válida se acepta.

El segundo defecto estaba en la consulta por identificador. La ruta recibe texto, pero el modelo compara con números, de modo que un identificador no numérico no coincidía con ningún registro y la respuesta resultaba ambigua. El controlador convierte ahora el parámetro a entero y exige un valor positivo, lo que distingue tres situaciones antes confundidas: parámetro mal formado, recurso inexistente y consulta correcta.

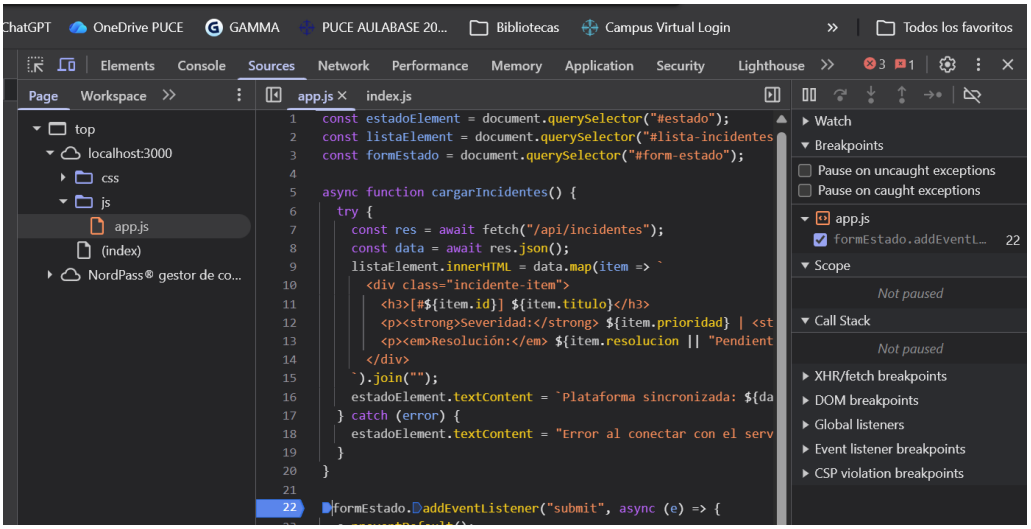


Figura 1. Reproducción del primer defecto. El punto de interrupción en el envío del formulario permite inspeccionar el valor del campo de resolución antes de que la petición salga del navegador.

```
// Defecto 1: el cierre exige una justificacion con contenido real
if (nuevoEstado === "CERRADO") {
  if (!resolucion || resolucion.trim().length < 5) {
    return { valido: false,
      razon: "Para cerrar un incidente es obligatorio ingresar una resolucion de al menos 5 caracteres" };
  }
```

```
}

// Defecto 2: el identificador se convierte y se valida antes de consultar el modelo
const id = parseInt(req.params.id, 10);
if (isNaN(id) || id <= 0) {
  return res.status(400).json({ error: "Parametro invalido",
    detalle: "El identificador del incidente debe ser un numero entero positivo" });
}
const incidente = IncidenteModel.obtenerPorId(id);
if (!incidente) return res.status(404).json({ error: "No encontrado", /* ... */ });
```

Listing 1: `src/utils/estado.js` y `src/controllers/incidente.controller.js` – las dos correcciones

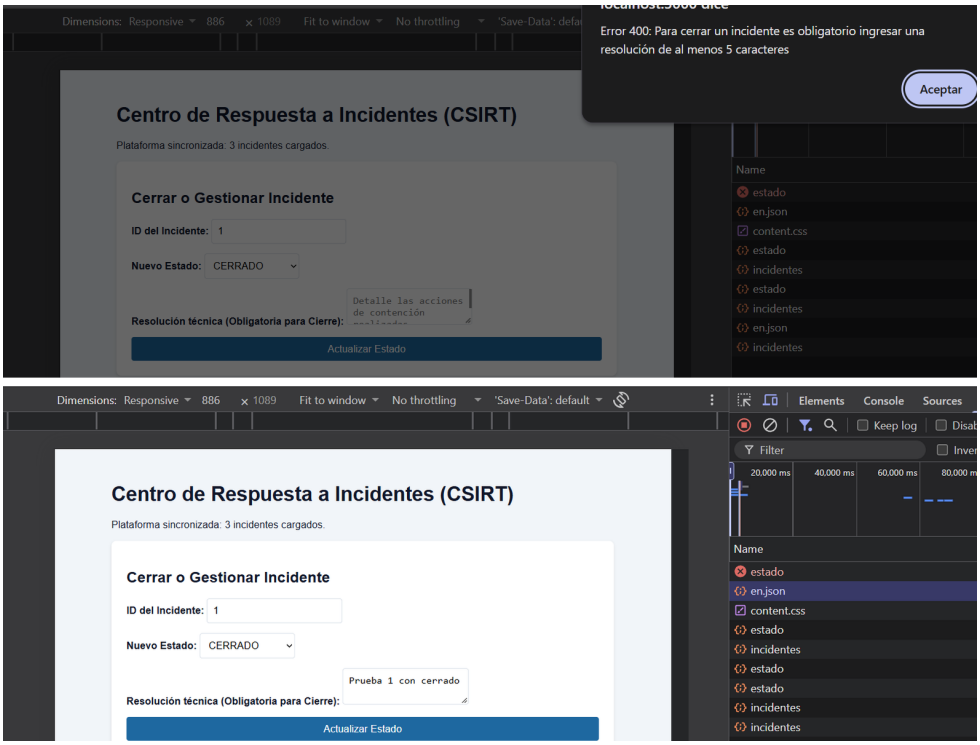


Figura 2. Comportamiento tras la corrección del primer defecto. Arriba, el cierre con resolución vacía se rechaza; abajo, el mismo cierre con una justificación válida se acepta y la tarjeta muestra el texto registrado.

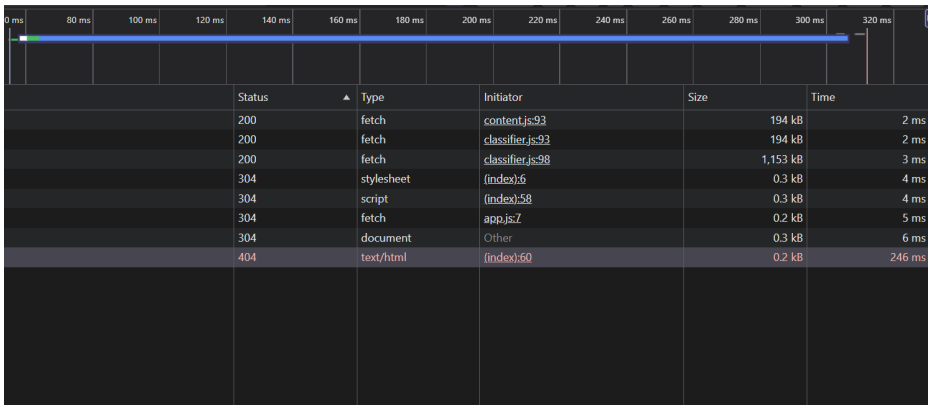


Figura 3. Defecto adicional observado en el panel de red: la ruta del recurso de imagen no resolvía y la petición devolvía un error, por lo que el diagrama no llegaba a mostrarse. La ruta se corrigió en el marcado.

3. Pruebas unitarias

Las reglas de transición viven en un módulo independiente, sin dependencias del servidor ni del navegador, por lo que se verifican en milisegundos y el fallo señala la regla exacta. La suite cubre el cierre con

justificación vacía y con justificación válida, la reactivación de un incidente cerrado, un estado de destino desconocido y una transición ordinaria. El primer caso reproduce el defecto corregido, de modo que ese error queda protegido ante cambios futuros. La cobertura del módulo es completa, aunque conviene precisar que mide líneas ejecutadas de ese archivo y no la calidad de las comprobaciones.



Figura 4. Ejecución de la suite y del reporte de cobertura. Los cinco casos aprueban y el módulo de reglas queda cubierto por completo en sentencias, ramas, funciones y líneas.

4. Análisis estático

El analizador estático se configuró para exigir comparaciones estrictas, prohibir declaraciones mutables innecesarias y señalar variables sin uso. El reporte inicial arrojó tres hallazgos: una colección declarada como variable reasignable que nunca se reasigna y dos parámetros capturados en bloques de error que nadie utilizaba. Tras atenderlos, la ejecución termina sin errores ni advertencias.

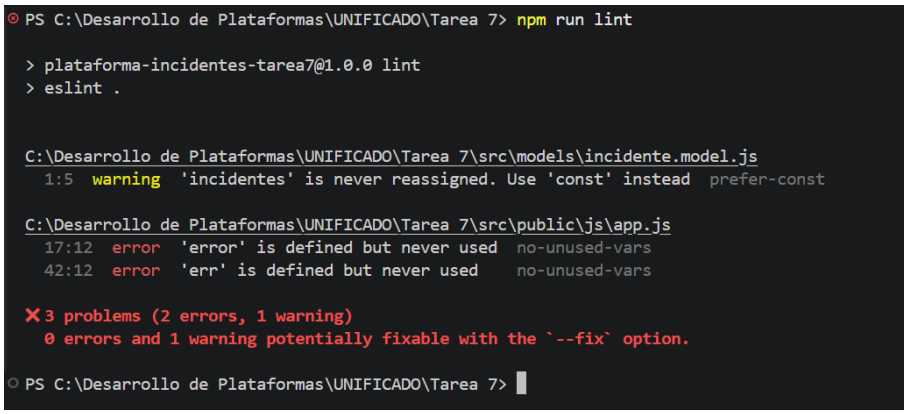


Figura 5. Reporte inicial del analizador: dos errores por variables declaradas y no usadas y una advertencia por una colección que podía declararse constante.



Figura 6. Ejecución posterior a las correcciones: el analizador termina con código de salida cero, sin errores ni advertencias.

5. Rendimiento: mediciones antes y después

Las tres mediciones se tomaron sobre la misma página, con el mismo navegador y la caché vacía. En un servidor local los tiempos quedan por debajo del milisegundo y no discriminan entre versiones, así que el indicador comparable es el volumen transferido.

La imagen del diagrama era el recurso dominante. El equipo la había convertido a un formato moderno y obtuvo una reducción apreciable, pero el archivo conservaba su resolución original mientras la página lo muestra a trescientos píxeles. Redimensionarlo al tamaño en que realmente se presenta lo reduce muy por debajo de la versión convertida.

La respuesta del listado transportaba campos que el tablero no usa. El modelo ya ofrecía una proyección reducida, de modo que bastó conectarla al controlador. La respuesta también carecía de instrucción de almacenamiento, por lo que el navegador volvía a pedirla en cada recarga; el servidor emite ahora una validez breve. Sumadas las tres medidas, la carga completa pasa de doscientos diecisiete kilobytes a veintisiete.

Tabla 2. Las tres mediciones, con su valor antes y después de la intervención

Medición	Antes	Después	Intervención aplicada
Imagen	343.7 KB	21.9 KB	Formato moderno y redimensionado al tamaño en que se muestra
Consulta	498 B	381 B	Proyección conectada al controlador, conservando los campos que la vista usa
Solicitud de red	sin caché	`max-age=60`	Instrucción de almacenamiento activada en el listado
Carga completa	**222 484 B**	**27 513 B**	**Reducción del 87.6 %**

Nota. Elaboración propia. El peso de la imagen se contrasta contra el archivo original en formato sin pérdida.



Figura 7. Composición de la transferencia en ambas versiones. El recurso de imagen concentraba prácticamente todo el peso de la página. Fuente: elaboración propia.

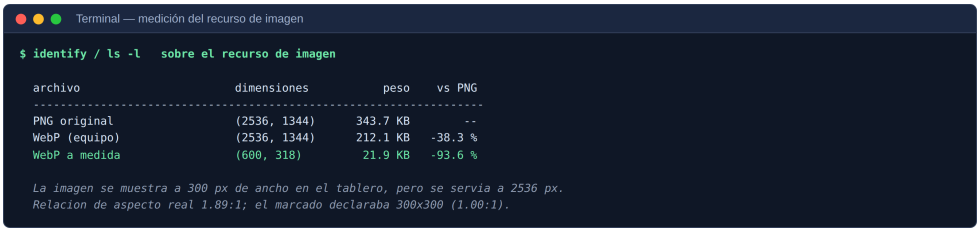


Figura 8. Medición del recurso de imagen. La conversión de formato aporta una reducción del 38.3 %; ajustar además la resolución al tamaño de presentación la lleva al 93.6 %.

6. Refactorización y documentación

La estructura separa la vista, las rutas, el controlador, el modelo y las reglas de negocio. Esa separación es la que hace posible probar las transiciones sin levantar el servidor y la que permite localizar un fallo por capa. Tras los cambios, las cinco pruebas siguen aprobando y el analizador sigue limpio, lo que constituye la verificación de que no hubo regresiones. El manual del repositorio se actualizó para reflejar el estado real de cada mejora.

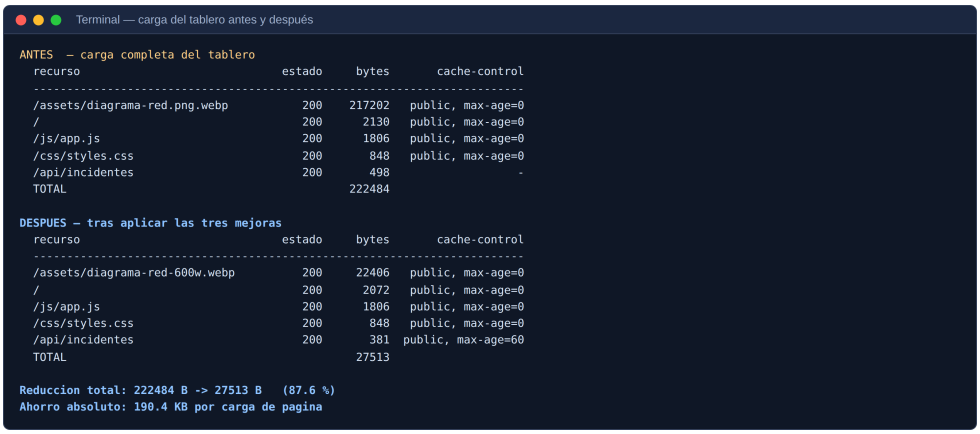


Figura 9. Recursos transferidos en la carga del tablero antes y después de aplicar las tres medidas, con el código de estado y la instrucción de almacenamiento de cada respuesta.

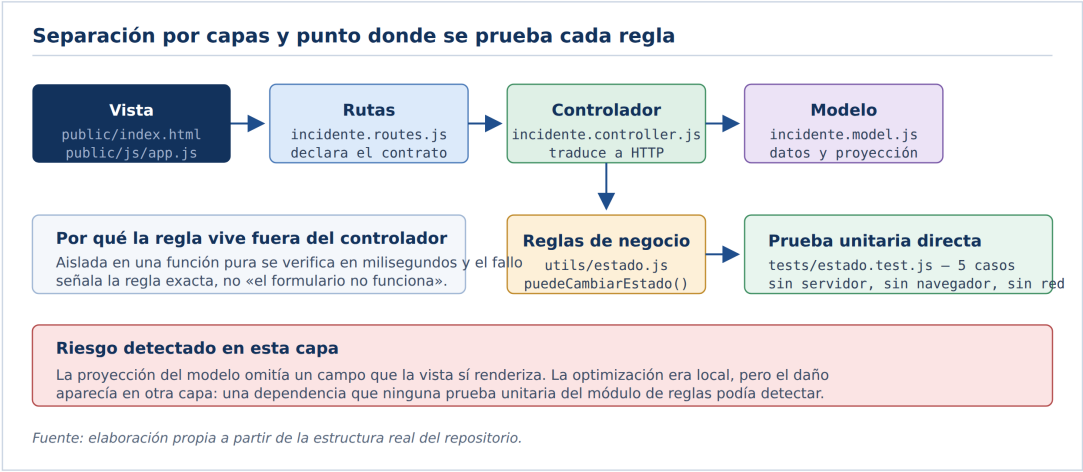


Figura 10. Separación por capas y punto en que se verifica cada regla. Fuente: elaboración propia.



Figura 11. Tablero tras la intervención. La vista conserva el mismo comportamiento y el diagrama se muestra con idéntica calidad aparente pese a pesar una décima parte.

7. Hallazgos de la auditoría

La revisión detectó una discrepancia entre lo que el manual declaraba y lo que el código ejecutaba. Dos de las tres optimizaciones figuraban como aplicadas, pero la proyección reducida nunca se invocaba desde el controlador y la instrucción de almacenamiento permanecía comentada. El informe redactado por el equipo sí lo advertía, de modo que ambos documentos se contradecían dentro de la misma entrega.

Más relevante es lo que se halló al conectar esa proyección: omitía el campo de resolución, que la vista sí renderiza. Aplicada tal como estaba escrita, habría mostrado todos los incidentes como pendientes, incluidos los cerrados con justificación. Es el caso que advierte la unidad, el de una optimización local que perjudica otra parte, y ninguna prueba del módulo de reglas podía detectarlo. La proyección se corrigió para conservar ese campo.

```
// ANTES -- definida en el modelo pero nunca invocada; ademas omite "resolucion"
obtenerResumen: () => incidentes.map(({ id, titulo, prioridad, estado }) =>
                                     ({ id, titulo, prioridad, estado })),

listar: (req, res) => {
  const lista = IncidenteModel.obtenerTodos();            // devuelve todos los campos
  // res.set("Cache-Control", "public, max-age=60");      // comentado
  return res.status(200).json(lista);
},

// DESPUES -- conserva el campo que la vista renderiza y queda conectada
obtenerResumen: () => incidentes.map(({ id, titulo, prioridad, estado, resolucion }) =>
                                     ({ id, titulo, prioridad, estado, resolucion })),

listar: (req, res) => {
  const lista = IncidenteModel.obtenerResumen();
  res.set("Cache-Control", "public, max-age=60");
  return res.status(200).json(lista);
},
```

Listing 2: La proyeccion antes y despues de la correccion, y su conexion desde el controlador

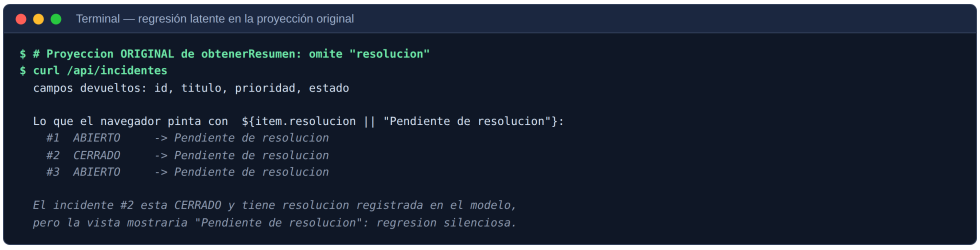


Figura 12. Demostración de la regresión latente. Con la proyección original, el incidente cerrado con justificación registrada se habría mostrado como pendiente en el tablero.



Figura 13. Revalidación condicional. El servidor ya emitía una etiqueta de entidad, de modo que una petición condicional devuelve una respuesta sin cuerpo; la instrucción de almacenamiento evita además el viaje de ida y vuelta.

8. Limitaciones

Quedan asuntos abiertos. Las dimensiones declaradas en el marcado no coincidían con la proporción real de la imagen y un estilo en línea las anulaba, lo que neutralizaba su propósito de evitar desplazamientos de contenido. Las pruebas cubren el módulo de reglas, no el controlador ni las rutas. Los incidentes residen en memoria y se pierden al reiniciar. Finalmente, este proyecto no incorpora la autenticación ni la persistencia construidas en las unidades anteriores.

Extensión del cuerpo redactado: 846 palabras (secciones 1 a 8, sin contar portada, encabezados, tablas, pies de figura, listados de código ni referencias).

Referencias Bibliográficas

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

Chrome for Developers. (s. f.). *Debug JavaScript*. Google. `https://developer.chrome.com/docs/devtools/javascript/`

ESLint. (s. f.). *Configure ESLint*. OpenJS Foundation. `https://eslint.org/docs/latest/use/configure/`

Feathers, M. (2004). *Working effectively with legacy code*. Prentice Hall.

Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2.ª ed.). Addison-Wesley.

Google. (s. f.). *Optimize Cumulative Layout Shift*. web.dev. `https://web.dev/articles/optimize-cls`

Grigorik, I. (2013). *High performance browser networking*. O'Reilly Media.

Jest. (s. f.). *Getting started*. OpenJS Foundation. `https://jestjs.io/docs/getting-started`

MDN Web Docs. (2025). *HTTP caching*. Mozilla. `https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching`

Pontificia Universidad Católica del Ecuador. (s. f.). *Desarrollo en plataformas. Unidad 7: Pruebas y optimización* [Aula virtual]. EVA PUCE. `https://eva.puce.edu.ec/EV-202502/course/view.php?id=71`

Wagner, J. (s. f.). *Serve images in modern formats*. web.dev. `https://web.dev/articles/uses-webp-images`